

result for one query possibly used as an input for another. In addition, one may arbitrarily extend an existing LNN model with neurons representing new formulae to serve as a novel query.

C.1 Proof of Theorem 1

We shall now proof Theorem 1, or specifically that, given weighted nonlinear logic with monotonic $\neg$, $\oplus$, and f , Algorithm 3 converges to within ϵ in finite time for the propositional case.

Proof. All operations in weighted nonlinear logic are implemented in terms of $\neg$, $\oplus$, and f and are thus also monotonic functions of their inputs. Truth value bounds aggregation is likewise monotonic, always taking the tightest available bounds, as per Algorithms 1, 2, and 3. Because lower bounds can only increase, but have a maximum value of 1, and upper bounds can only decrease, but have a minimum value of 0, each sequence of updates for a given bound must be constitute a Cauchy sequence; otherwise, there would have to exist some step size $\delta > 0$ by which some truth value bound updates an infinite number of times, but this would clearly push the bound past its limit of 1 or 0. Accordingly, after some finite number of iterations of Algorithm 3, all truth value bounds will be within $\frac{\epsilon}{n}$ of the end point of their Cauchy sequence. For n total lower and upper bounds, the sum of all such deviations will be at most ϵ . $\square$

This proof does not apply to FOL because, unlike the propositional case, FOL can introduce lower and upper bounds at new predicate groundings throughout evaluation. For example, a successor function can produce an infinite number of predicate groundings; while each of these constitutes its own Cauchy sequence and thus necessarily converges independently of the others, the entire system may never converge. This result is expected given the well known undecidable nature of FOL.

D Proof of Theorem 2

To prove Theorem 2, let us first give an expanded description of the LNN variant from the first paragraph of Section 5. It is mathematically equivalent to that paragraph, and the expanded bound-update equations for various logical connectives will facilitate the proof.

Consider a bipartite graph $G = \langle V_1 \cup V_2, E \rangle$, where each node in V_1 represents a formula and each node in V_2 represents a connective. Nodes in V_2 have the following types: NOT, IMPLIES, AND-2, OR-2, AND-3, OR-3, $\dots$. A NOT node always has a degree of 2. An IMPLIES node always has a degree of 3. An AND- k or OR- k node always has a degree of $k + 1$. For a node $v \in V_1 \cup V_2$, let d_v denote its degree and let $n_{v,1}, \dots, n_{v,d_v}$ denote its neighbors.

Define two functions $L : V_1 \rightarrow [0, 1]$ and $U : V_1 \rightarrow [0, 1]$. Each node $v \in V_2$ is annotated with $2d_v$ functions:

$$\begin{aligned} \tilde{L}_{L,U,v,1} &= f_{v,1}(L(n_{v,2}), \dots, L(n_{v,d_v}), U(n_{v,2}), \dots, U(n_{v,d_v})) \\ \tilde{U}_{L,U,v,1} &= g_{v,1}(L(n_{v,2}), \dots, L(n_{v,d_v}), U(n_{v,2}), \dots, U(n_{v,d_v})) \\ \tilde{L}_{L,U,v,2} &= f_{v,2}(L(n_{v,1}), L(n_{v,3}), \dots, L(n_{v,d_v}), U(n_{v,1}), U(n_{v,3}), \dots, U(n_{v,d_v})) \\ \tilde{U}_{L,U,v,2} &= g_{v,2}(L(n_{v,1}), L(n_{v,3}), \dots, L(n_{v,d_v}), U(n_{v,1}), U(n_{v,3}), \dots, U(n_{v,d_v})) \\ &\dots \\ \tilde{L}_{L,U,v,d_v} &= f_{v,d_v}(L(n_{v,1}), \dots, L(n_{v,d_v-1}), U(n_{v,1}), \dots, U(n_{v,d_v-1})) \\ \tilde{U}_{L,U,v,d_v} &= g_{v,d_v}(L(n_{v,1}), \dots, L(n_{v,d_v-1}), U(n_{v,1}), \dots, U(n_{v,d_v-1})) \end{aligned}$$

Without loss of generality, let's assume an indexing scheme for neighbors of $v \in V_2$: if v has type AND- k , $n_{v,k+1} = n_{v,1} \wedge \dots \wedge n_{v,d_k}$; if v has type OR- k , $n_{v,k+1} = n_{v,1} \vee \dots \vee n_{v,k}$; if v has type IMPLIES, $n_{v,3} = n_{v,1} \rightarrow n_{v,2}$.

With different choices of $\tilde{L}$'s and $\tilde{U}$'s, LNN can implement different flavors of real-valued logic. The variant in Section 5 uses the following choice of $\tilde{L}$'s and $\tilde{U}$'s, grouped by types of node $v \in V_2$.

- NOT

$$\tilde{L}_{L,U,v,1} = 1 - U(n_{v,2}) \quad (25)$$

$$\tilde{U}_{L,U,v,1} = 1 - L(n_{v,2}) \quad (26)$$

$$\tilde{L}_{L,U,v,2} = 1 - U(n_{v,1}) \quad (27)$$

$$\tilde{U}_{L,U,v,2} = 1 - L(n_{v,1}) \quad (28)$$

- AND- k

For $i = 1, \dots, k$:

$$\tilde{L}_{L,U,v,i} = L(n_{v,k+1}) \quad (29)$$

$$\tilde{U}_{L,U,v,i} = \min \left(1, U(n_{v,k+1}) + \sum_{1 \leq j \leq k, j \neq i} (1 - L(n_{v,j})) \right) \quad (30)$$

$$\tilde{L}_{L,U,v,k+1} = \max \left(0, 1 - \sum_{j=1}^k (1 - L(n_{v,j})) \right) \quad (31)$$

$$\tilde{U}_{L,U,v,k+1} = \min_{j=1}^k U(n_{v,j}) \quad (32)$$

- OR- k

For $i = 1, \dots, k$:

$$\tilde{L}_{L,U,v,i} = \max \left(0, L(n_{v,k+1}) - \sum_{1 \leq j \leq k, j \neq i} U(n_{v,j}) \right) \quad (33)$$

$$\tilde{U}_{L,U,v,i} = U(n_{v,k+1}) \quad (34)$$

$$\tilde{L}_{L,U,v,k+1} = \max_{j=1}^k L(n_{v,j}) \quad (35)$$

$$\tilde{U}_{L,U,v,k+1} = \min \left(1, \sum_{j=1}^k U(n_{v,j}) \right) \quad (36)$$

- IMPLIES

$$\tilde{L}_{L,U,v,1} = 1 - U(n_{v,3}) \quad (37)$$

$$\tilde{U}_{L,U,v,1} = \min(1, 1 + U(n_{v,2}) - L(n_{v,3})) \quad (38)$$

$$\tilde{L}_{L,U,v,2} = \max(0, L(n_{v,1}) + L(n_{v,3}) - 1) \quad (39)$$

$$\tilde{U}_{L,U,v,2} = U(n_{v,3}) \quad (40)$$

$$\tilde{L}_{L,U,v,3} = \max(1 - U(n_{v,1}), L(n_{v,2})) \quad (41)$$

$$\tilde{U}_{L,U,v,3} = \min(1, 1 - L(n_{v,1}) + U(n_{v,2})) \quad (42)$$

With the same notations as in Section 5, initial knowledge of LNN inference is specified by a set of formulas $V_0 \subseteq V_1$ and two functions $L_0 : V_0 \rightarrow [0, 1]$ and $U_0 : V_0 \rightarrow [0, 1]$. The query formula σ is either an existing node in V_1 or a formula that is composed of existing nodes in V_1 . Without loss of generality, let's assume $\sigma \in V_1$, because otherwise we can first expand G by adding connectives until σ is included in V_1 .

Consider node v and one of its neighbors $n_{v,i}$, let $j_{v,i}$ denote v 's index among $n_{v,i}$'s neighbors. In other words, $n_{n_{v,i}, j_{v,i}} \equiv v$.

LNN inference, i.e., Algorithm 3 in the main paper, can be written as the following pseudo code, where the selection of v and i in the loop is determined by the upward and downward passes of Algorithms 1 and 2 in the main paper.

```

 $L(v) \leftarrow L_0(v), \forall v \in V_0;$ 
 $U(v) \leftarrow U_0(v), \forall v \in V_0;$ 
 $L(v) \leftarrow 0, \forall v \in V_1 \setminus V_0;$ 
 $U(v) \leftarrow 1, \forall v \in V_1 \setminus V_0;$ 
do until convergence:
  select  $v \in V_1$  and index  $i \in \{1, \dots, d_v\};$ 
   $L(v) \leftarrow \max \left( L(v), \tilde{L}_{L,U,n_{v,i},j_{v,i}} \right), \forall v \in V_1;$ 
   $U(v) \leftarrow \min \left( U(v), \tilde{U}_{L,U,n_{v,i},j_{v,i}} \right), \forall v \in V_1;$ 
return  $L(\sigma)$  and  $U(\sigma);$ 

```

To prove Theorem 2, we will start with following lemma which serves as a steppingstone.

Lemma 1. *For any functions $L : V_1 \rightarrow [0, 1]$ and $U : V_1 \rightarrow [0, 1]$, and for any $\phi \in V_1$ and any $i \in \{1, \dots, d_\phi\}$, define $\Gamma = \{(v, L(v), U(v)) \mid v \in V_1\}$ and define $\Gamma'_{\phi,i} = (\Gamma \setminus \{(\phi, L(\phi), U(\phi))\}) \cup \{(\phi, \max(L(\phi), \tilde{L}_{L,U,n_{\phi,i},j_{\phi,i}}), \min(U(\phi), \tilde{U}_{L,U,n_{\phi,i},j_{\phi,i}}))\}$. The following equality holds:*

$$P_\Gamma = P_{\Gamma'_{\phi,i}} \quad (43)$$

Proof of Lemma 1. Γ and $\Gamma'_{\phi,i}$ differ by exactly one sentence: the former contains $(\phi, L(\phi), U(\phi))$ while the latter contains $(\phi, \max(L(\phi), \tilde{L}_{L,U,n_{\phi,i},j_{\phi,i}}), \min(U(\phi), \tilde{U}_{L,U,n_{\phi,i},j_{\phi,i}}))$. Since $L(\phi) \leq \max(L(\phi), \tilde{L}_{L,U,n_{\phi,i},j_{\phi,i}})$ and $U(\phi) \geq \min(U(\phi), \tilde{U}_{L,U,n_{\phi,i},j_{\phi,i}})$, by definition any model of $\Gamma'_{\phi,i}$ must be a model of Γ . In other words, we have $P_{\Gamma'_{\phi,i}} \subseteq P_\Gamma$. Therefore, in order to prove Lemma 1, we only need to show $P_\Gamma \subseteq P_{\Gamma'_{\phi,i}}$. It is trivially true if $P_\Gamma = \emptyset$, and therefore it suffices to show that $p \in P_{\Gamma'_{\phi,i}}$ for any $p \in P_\Gamma$. By definition, it is equivalent to show the following two inequalities for any $p \in P_\Gamma$:

$$p(S_\phi) \leq \tilde{U}_{L,U,n_{\phi,i},j_{\phi,i}} \quad (44)$$

$$p(S_\phi) \geq \tilde{L}_{L,U,n_{\phi,i},j_{\phi,i}} \quad (45)$$

Because the right-hand side of (44) and (45) may come from any of (25)–(42), we'll now prove (44) or (45) for all eighteen possibilities:

- The right-hand side of (45) comes from (25) or (27). By definition, $n_{\phi,i}$ is a node of type NOT, and let $\gamma \triangleq n_{n_{\phi,i},3-j_{\phi,i}}$ denote the other neighbor of $n_{\phi,i}$. By definition, we have $\gamma = \neg\phi$, $\tilde{L}_{L,U,n_{\phi,i},j_{\phi,i}} = 1 - U(\gamma)$, and $p(S_\gamma) \leq U(\gamma)$. Therefore,

$$p(S_\phi) = p(S_{\neg\gamma}) = 1 - p(S_\gamma) \geq 1 - U(\gamma) = \tilde{L}_{L,U,n_{\phi,i},j_{\phi,i}} \quad (46)$$

Hence (45) is true in this scenario.

- The right-hand side of (44) comes from (26) or (28). By definition, $n_{\phi,i}$ is a node of type NOT, and let $\gamma \triangleq n_{n_{\phi,i},3-j_{\phi,i}}$ denote the other neighbor of $n_{\phi,i}$. By definition, we have $\gamma = \neg\phi$, $\tilde{U}_{L,U,n_{\phi,i},j_{\phi,i}} = 1 - L(\gamma)$, and $p(S_\gamma) \geq L(\gamma)$. Therefore,

$$p(S_\phi) = p(S_{\neg\gamma}) = 1 - p(S_\gamma) \leq 1 - L(\gamma) = \tilde{U}_{L,U,n_{\phi,i},j_{\phi,i}} \quad (47)$$

Hence (44) is true in this scenario.

- The right-hand side of (45) comes from (29). By definition, $n_{\phi,i}$ is a node of type AND- k , and let $\gamma \triangleq n_{n_{\phi,i},k+1}$ and $\xi_1, \dots, \xi_{k-1}$ denote the other neighbors of $n_{\phi,i}$. By definition, we have $\gamma = \phi \wedge \xi_1 \wedge \dots \wedge \xi_{k-1}$, $\tilde{L}_{L,U,n_{\phi,i},j_{\phi,i}} = L(\gamma)$, and $p(S_\gamma) \geq L(\gamma)$. It is straightforward to verify that $S_\gamma \subseteq S_\phi$. Therefore,

$$p(S_\phi) \geq p(S_\gamma) \geq L(\gamma) = \tilde{L}_{L,U,n_{\phi,i},j_{\phi,i}} \quad (48)$$

Hence (45) is true in this scenario.